

# NetworkPolicy & Protocols

*TCP, UDP, SCTP, ICMP, and where the standard API stops*

**L3**

IP / CIDR

**L4**

Protocol + port

**L7**

HTTP, DNS, Kafka

# Which protocols can a NetworkPolicy name?

```
ports:  
- protocol: TCP    # or UDP, SCTP  
  port: 80
```

TCP

Supported

UDP

Supported

SCTP

Supported

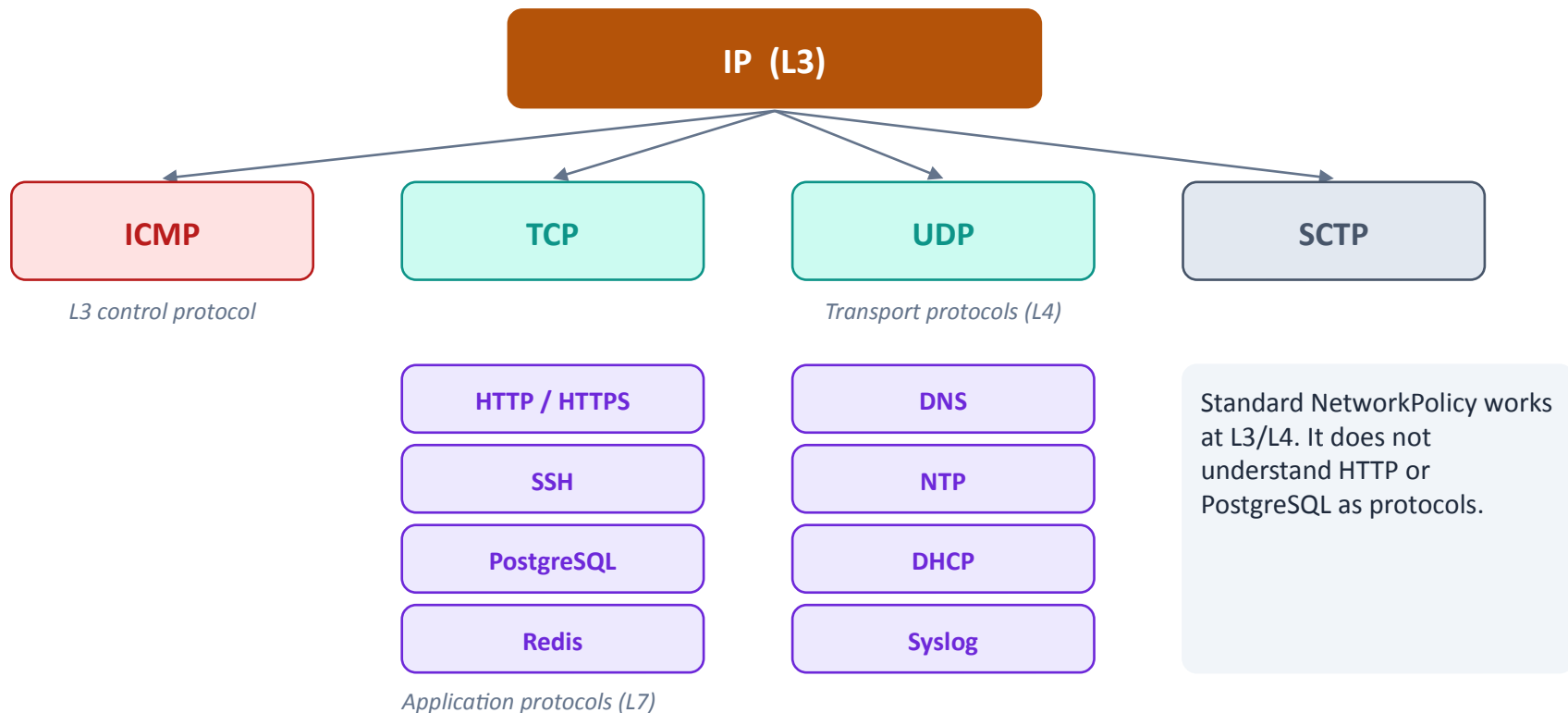
ICMP

Not in standard API

## Why ICMP is different

- The ports field is built around transport protocols that have port numbers
- ICMP has no ports: it uses message types (echo request, unreachable, time exceeded)
- So "protocol: ICMP, port: 8" cannot be expressed
- Calico and Cilium add their own policy APIs that can match ICMP types

# Protocols sit at different layers



DO NOT CONFUSE

# Protocol is not the same as port

## TCP

*identified by port numbers*

:80 HTTP

:443 HTTPS

:22 SSH

:5432 PostgreSQL

## UDP

*identified by port numbers*

:53 DNS

:123 NTP

:514 Syslog

:8472 VXLAN

## ICMP

*identified by message types*

8 Echo Request

0 Echo Reply

3 Dest. Unreachable

11 Time Exceeded

"Allow ICMP port 80" does not make sense: ICMP has types, not ports.

# 01

## Standard NetworkPolicy

*What `networking.k8s.io/v1` can and cannot express*

# Allow web traffic to nginx

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-web
spec:
  podSelector:
    matchLabels:
      app: nginx
  policyTypes:
    - Ingress
  ingress:
    - ports:
        - protocol: TCP
          port: 80
        - protocol: TCP
          port: 443
```

## podSelector

Which Pods this policy protects

## policyTypes: Ingress

Selected Pods now deny all ingress not allowed below

## ingress[].ports

Protocol + port pairs that are allowed

## No from:

Any source may use those ports

# How the ports field behaves

## protocol omitted

Defaults to TCP

```
- port: 80           # means TCP 80
```

## Named port

Matches the container port name

```
- port: http         # containerPort name
```

## Port range

endPort (stable since v1.25)

```
- port: 30000  
  endPort: 32767
```

## ports omitted

ALL ports and ALL protocols from the peer

```
- from:  
  - podSelector: {...} # no ports
```

# You cannot write this

```
ports:  
- protocol: ICMP  
  port: 8
```

**Rejected: Unsupported value "ICMP"**

Supported values: "SCTP", "TCP", "UDP"

The API models L4 ports only. Matching ICMP types needs a CNI-specific policy resource.

## ICMP has no port to match

Echo Request (8)

Echo Reply (0)

Destination Unreachable (3)

Time Exceeded (11)

... and more

# ICMP is blocked or allowed as a side effect

## Case 1: port-restricted rule

```
ingress:
- from:
  - podSelector:
      matchLabels: {role: client}
  ports:
  - protocol: TCP
    port: 80
```

curl :80 works

ping blocked

## Case 2: no ports listed

```
ingress:
- from:
  - podSelector:
      matchLabels: {role: client}
  # no ports: all protocols
```

curl works

ping works

Standard API: ICMP is all-or-nothing per peer. "Only ping" is not expressible.

# Allow-lists only: there is no "deny" rule

## How rules combine

Pod not selected by any policy

**Everything allowed**

Pod selected (Ingress)

**Default deny ingress**

Matching rules from all policies

**Union of allows**

## Default deny all ingress in a namespace

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
spec:
  podSelector: {}
  policyTypes:
    - Ingress
```

To "deny TCP 8080", simply do not allow it. Explicit Deny actions exist only in Calico / Cilium.

# Selecting sources by IP / CIDR

```
ingress:
- from:
  - ipBlock:
      cidr: 10.10.0.0/16
      except:
        - 10.10.5.0/24
  ports:
    - protocol: TCP
      port: 443
```

## Meaning

**From 10.10.0.0/16**

**except 10.10.5.0/24**

**only TCP 443**

*Use ipBlock for traffic outside the cluster. For Pods, use podSelector / namespaceSelector: Pod IPs change.*

**L3 (who) and L4 (what) combine in the same rule: both must match.**

# Default-deny egress breaks DNS

```
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: kube-system
      podSelector:
        matchLabels: {k8s-app: kube-dns}
    ports:
    - {protocol: UDP, port: 53}
    - {protocol: TCP, port: 53}
```

## Why both protocols

- DNS uses UDP 53 normally
- Falls back to TCP 53 for large answers
- Allow only UDP and some lookups fail intermittently
- Symptom: "could not resolve host" after applying egress policy

# 02

## CNI-Specific Policy

*Calico and Cilium extend beyond the standard API*

# Allow ping, and only ping

```
apiVersion: projectcalico.org/v3
kind: NetworkPolicy
metadata:
  name: allow-ping
  namespace: default
spec:
  selector: app == 'nginx'
  types: [Ingress]
  ingress:
    - action: Allow
      protocol: ICMP
      icmp:
        type: 8          # Echo Request
```

## What Calico adds

**protocol: ICMP / ICMPv6**

plus TCP, UDP, SCTP and numeric protocols

**icmp / notICMP**

match or exclude by type and code

**action**

Allow, Deny, Log, Pass

**order**

explicit rule priority

**GlobalNetworkPolicy**

cluster-wide rules

# L3, L4 and L7 in one API

**L3**

Who

fromEndpoints (labels)

fromCIDR

fromEntities: world, host

toFQDNs

**L4**

What

toPorts: TCP / UDP / SCTP

icmps: ICMP types

port ranges

**L7**

Which request

HTTP method, path, headers

DNS names queried

Kafka topics, gRPC

## Cilium ICMP example

```
ingress:  
- fromEndpoints: [{matchLabels: {role: client}}]  
  icmps:  
  - fields: [{type: 8, family: IPv4}]
```

Also: ingressDeny / egressDeny for  
explicit denies

# Allow HTTP GET, block everything else

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: nginx-get-only
spec:
  endpointSelector:
    matchLabels: {app: nginx}
  ingress:
    - fromEndpoints:
      - matchLabels: {role: client}
      toPorts:
        - ports:
            - {port: "80", protocol: TCP}
          rules:
            http:
              - {method: GET, path: "/*.*"}
```

## Results

`curl -X GET :80/`

200 OK

`curl -X POST :80/`

403 Access denied

`curl :8080`

timeout (L4 drop)

`ping`

dropped

L4 policy sees "TCP 80". L7 policy sees "GET /index.html".

# What each policy engine can express

| Capability            | Standard API (K3s kube-router) | Calico     | Cilium    |
|-----------------------|--------------------------------|------------|-----------|
| TCP / UDP ports       | Yes                            | Yes        | Yes       |
| SCTP                  | API yes; CNI support varies    | Yes        | Yes       |
| Port ranges           | Yes (endPort)                  | Yes        | Yes       |
| IP / CIDR             | Yes (ipBlock)                  | Yes        | Yes       |
| ICMP by type          | No                             | Yes        | Yes       |
| Explicit deny         | No                             | Yes        | Yes       |
| Rule ordering         | No                             | Yes        | Deny wins |
| Cluster-wide policy   | No                             | Yes        | Yes       |
| FQDN egress           | No                             | Enterprise | Yes       |
| L7 (HTTP, DNS, Kafka) | No                             | Limited    | Yes       |

# 03

## Hands-on Lab

*Watch L3, L4 and L7 policy take effect step by step*

# One server, one client, three tests

```
# Pod B: server
kubectl run pod-b --image=nginx --labels=app=nginx

# Pod A: client with ping, curl, nc
kubectl run pod-a --image=nicolaka/netshoot \
  --labels=role=client -- sleep infinity

B=$(kubectl get pod pod-b -o jsonpath='{.status.podIP}')

# Tests (repeat after each policy)
kubectl exec pod-a -- ping -c2 -W2 $B
kubectl exec pod-a -- curl -s -m3 -o /dev/null -w "%{http_code}" $B
kubectl exec pod-a -- nc -zv -w3 $B 8080
```

## Read the failure mode

### Connection refused

Packet reached the Pod; nothing listens on that port

### Timeout

Packet silently dropped by policy

# Four policies, expected results

| Policy                          | Engine          | ping    | curl :80          | nc :8080 |
|---------------------------------|-----------------|---------|-------------------|----------|
| 0. No policy                    | any             | OK      | 200               | refused  |
| 1. Allow TCP 80 only            | Standard (K3s)  | blocked | 200               | timeout  |
| 2. Allow all from role=client   | Standard (K3s)  | OK      | 200               | refused  |
| 3. Policy 1 + allow ICMP type 8 | Calico / Cilium | OK      | 200               | timeout  |
| 4. HTTP GET only (L7)           | Cilium          | blocked | GET 200, POST 403 | timeout  |

Policy 2 shows the standard-API ICMP trick: allowing a peer with no ports lets ping through, but also opens every port. Policy 3 needs a CNI API to allow ping alone.

# Standard API policies for K3s

## Policy 1: TCP 80 only

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: p1-tcp80}
spec:
  podSelector:
    matchLabels: {app: nginx}
  policyTypes: [Ingress]
  ingress:
    - ports:
      - {protocol: TCP, port: 80}
```

## Policy 2: everything from client

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: p2-client-all}
spec:
  podSelector:
    matchLabels: {app: nginx}
  policyTypes: [Ingress]
  ingress:
    - from:
      - podSelector:
          matchLabels: {role: client}
```

## KEY TAKEAWAYS

# Know which layer you are filtering

### Standard

NetworkPolicy matches TCP, UDP, SCTP + ports. Allow-lists only.

### ICMP

has types, not ports. Standard API allows it only via "all protocols".

### Calico

adds ICMP types, explicit deny, ordering and global policy.

### Cilium

adds L7 rules for HTTP, DNS and Kafka, plus FQDN egress.